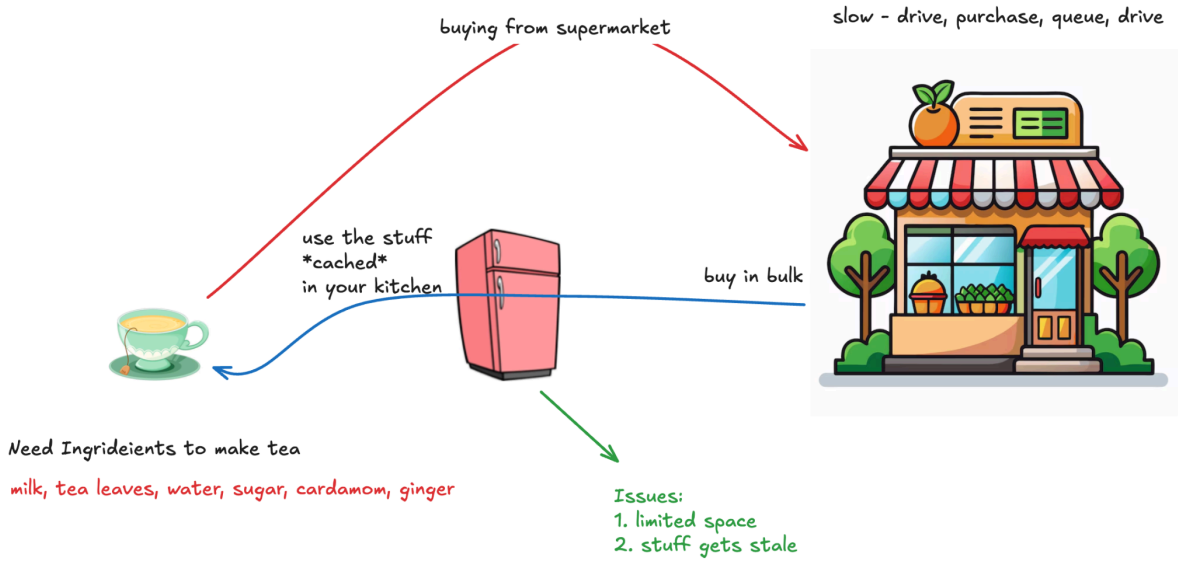
For any suggestions or feedback regarding these notes, please contact [Pragy Agarwal]

Need for Caching

Making Tea



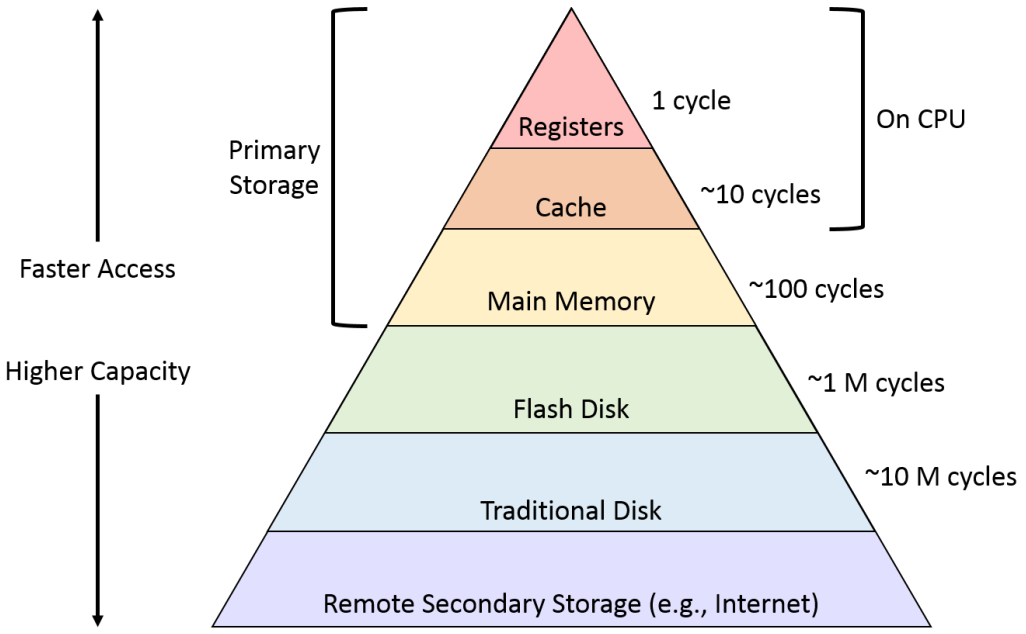
Human Brain

1. **Working memory:** extremely fast, only hold 4-7 items, volatile
2. **Short term memory:** very fast, hold thousands of items, volatile (slightly longer duration - up to weeks)
3. **Long term memory:** extremely slow, practically unlimited, permanent
    - when you go to sleep (REM), your brain consolidates your short-term memory into long-term memory.

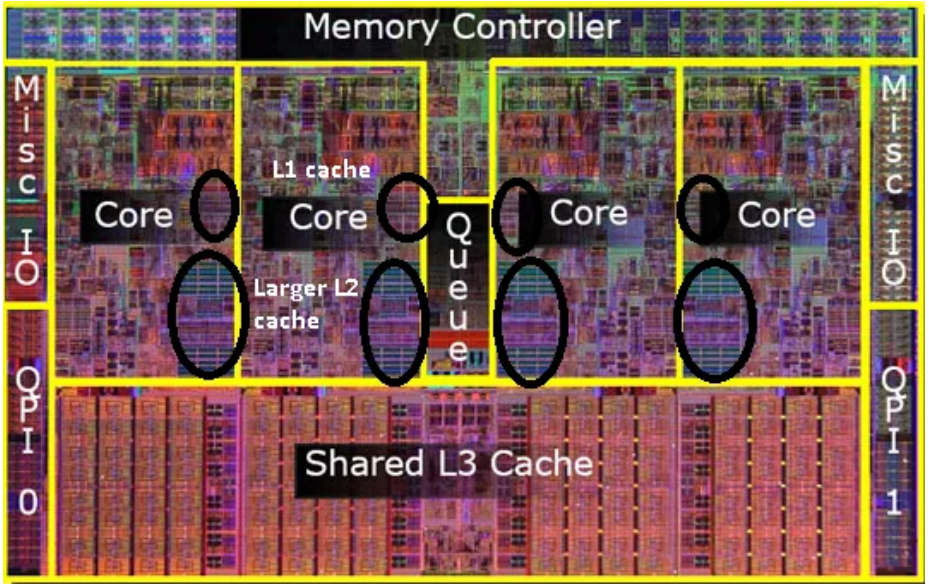
Even biological memory is hierarchical!

Computer Memory

- **CPU Registers**
  - this is what the CPU actually interacts with
  - very small (few bytes)
  - cpu registers work at clock-tick speed
    - `0.2 ns per access`
- **Cache** (inside the CPU) - L1, L2, L3
  - L1, L2: exist per core
  - L3: shared across all CPU cores
- **Main memory** (RAM) is almost `500x - 1000x` slower than CPU registers
  - `100ns` (traditionally)
  - modern DDR4 RAMs have a random read latency of `~10ns`
- **HDD** (magnetic disk drive): `100,000x` slower than the RAM



The Memory Hierarchy



How storage works

1. **Latency numbers:** [Latency Numbers Every Programmer Should Know]
2. **Magnetic:** How do Hard Disk Drives Work? 💻💿🛠️
3. **Solid State:** How do SSDs Work? | How does your Smartphone store data? | I…
4. **RAM:** How does Computer Memory Work? 💻🛠️
5. (extra) **Brain:**
    a. Information Storage and the Brain: Learning and Memory
    b. How We Make Memories: Crash Course Psychology #13
    c. Tools to Enhance Working Memory & Attention
6. **Optimizing code for efficient cache usage:**
   CppCon 2016: Timur Doumler “Want fast C++? Know your hardware!"

Memory is always hierarchical

- Larger memory storages will be slower compared to smaller ones
    - if you have to produce a lot of something, you will use cheaper technology
    - because registers of the size of RAM will be way too expensive
- Therefore, you have a need for caching

Web Memory Hierarchy

Browser (client side)

Not accessible to the frontend dev

1. **DNS cache** (chrome://net-internals/#dns)
2. **File/Media Cache**

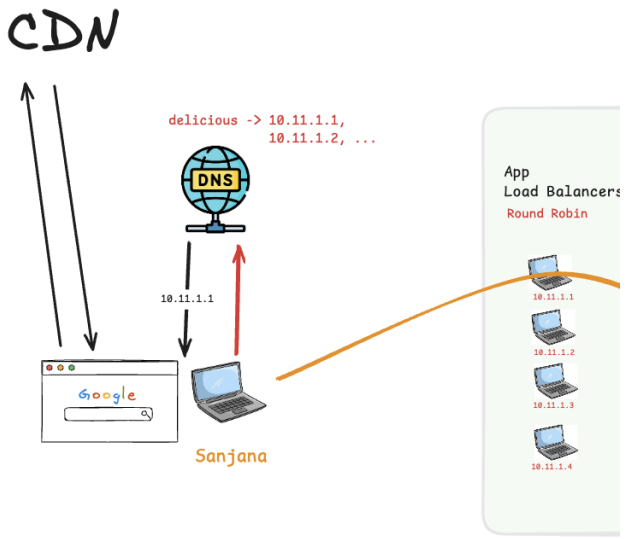
Accessible to the frontend dev

3. **Cookies / Session storage:** `1KB - 10KB` of stuff inside the cookies
4. **Localstorage:** up to `10MB`
5. **IndexedDB:** up to `10GB`

Content Delivery Network (CDN)

***Akamai**, Fastly, **Cloudflare**, **Cloudfront**, Cloudinary, ...*
CDNs are the backbone of the internet

CDNs are neither client side, nor backend side. They're a 3rd party service (like DNS)



Q: What type of data does a CDN cache?

Large `(> 10kb)`, mostly static *(changes rarely)* data
- multimedia: images, audio, videos, pdfs, zip, ...
- code: js, html, css

Any user-content (images/videos/reels/avatars uploaded by the users) will be served via CDNs

Q: Are CDNs databases?

No — they're a cache. They don't "store" the data, they just temporarily hold it to "serve" it to the clients.

Q: Can our backend servers access the CDN?

No. CDNs are accessed only by the clients

Q: What problem do the CDNs solve?

They reduce latency

Imagine that you're trying to watch a video (Scaler lecture recording)
Scaler servers are in Mumbai (South East Asia availability zone in AWS). Image that you're living in US.

What is the round-trip-time from US to India?

circumference of earth: `40,000 Km`

speed of light (in vacuum): `c = 299792458 m/s ~ 3*10^8 m/s`
but inside fiber optic cables, the speed of light is lesser: `~2 * 10^8 m/s`
round trip time for a packet to go from India to US and back:

```
40,000 KM / (2 * 10^8 m/s)
= 4 * 10^4 * 10^3 m   / (2 * 10^8 m/s)
= 4 / (20) seconds
= 200 ms
```

It takes at-least `200ms` for a packet to make the US–India–US trip (in reality, it will be closed to `500-600ms`)

This means that if you're fetching the data from far away, there will be higher latency.

Also, if you're fetching the data from far away, the bandwidth will be also be slower

1. multiple hops — your bandwidth is limited by the slowest server in the route the data takes
2. Intercontinental fiber optic cables handle trans-atlantic/trans-pacific data – they connect various continents & countries. All the network traffic across these continents travels over these cables
    - these cables are a bottleneck

What if instead of streaming from video from the Indian servers, you could get that video from a server near your home!

- very low latency (a few ms)
- bandwidth will be much higher

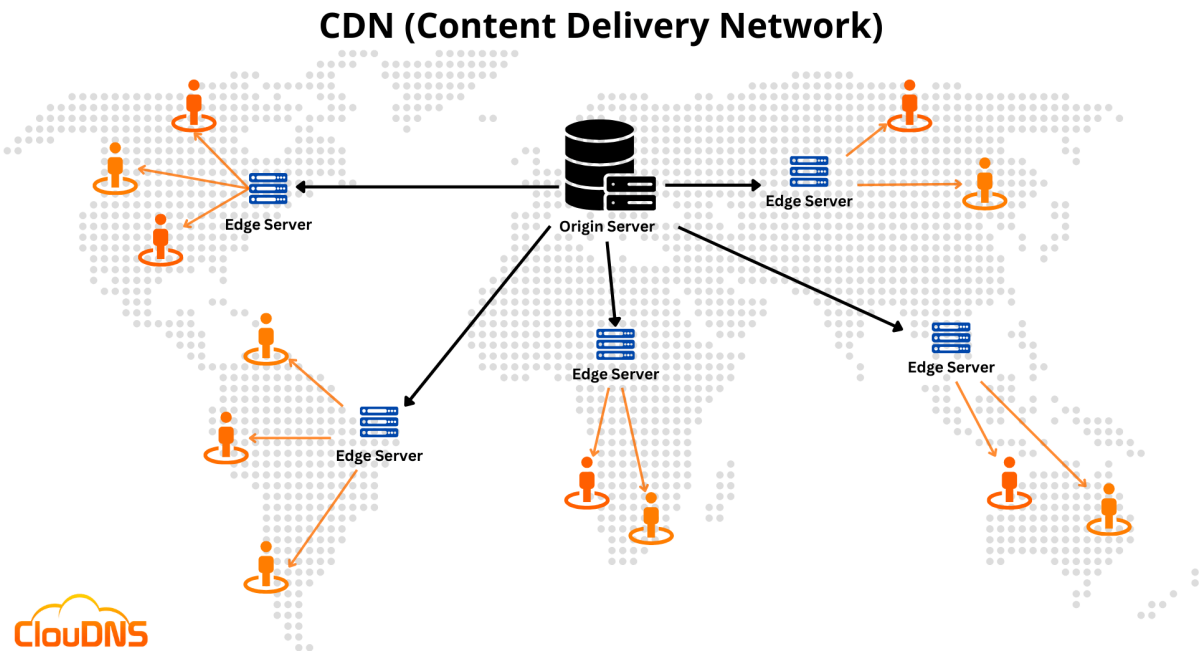
Edge: close to the user

Edge-node / edge-server: a server which is (physically/geographically) close to the user

If we could purchase 1,000,000 servers and place them all across the globe to act as caches, we could serve the data from a server closer to the user.

This is what CDNs do!
CDNs provide a global infrastructure of edge-nodes. They rent-out these servers to whoever wants to use it.

CDNs handle almost 70% of all the traffic on the internet!



Q. How can you find the IP address of the nearest CDN server (edge server)?

1. **GeoDNS:** special type of DNS that can resolve the closest IP address based on the user's geographical location
    - unfortunately, GeoDNS has low penetration
2. **AnyCast:** custom redirect that the CDNs provide to send you to the nearest server

How does the CDN get the data in the first place?

Imagine that Abdul uploads a video

- Abdul makes a request to Facebook's backend servers
- Starts uploading the video to their servers
- Facebook serves will store this video in a file storage (like S3)
- Facebook will store the metadata in some database (like Mongodb, like SQL)
- Facebook will hit the CDN servers and configure a URL
    - Facebook → Akamai
    - Please give me the CDN url for this database URL "s3.aws.com/1234/video.mp4"
    - CDN will store this URL and give a unique CDN URL corresponding to it "cdn.akamai.com/a3b2fd5.mp4"
    - CDN will store the mapping from its URL to the database URL internally

Imagine that Upinta wants to watch this video

- she will make a request to the facebook backend servers
- backend servers will return a HTML+JS+CSS page

```
<html>
<body>
    ~~<video src="s3.aws.com/1234/video.mp4" />~~
    <video src="cdn.akamai.com/a3b2fd5.mp4" />
</body>
</html>
```

- Upinta's browser will download the video from the URL embedded in the HTML file
    - if the embedded URL is the database URL, then Upinta might be located far away from the database servers — her download speed will be slow and her latency will be high

      `<video src="s3.aws.com/1234/video.mp4" />`
    - Instead, if the embedded URL is the CDN URL, then Upinta will be able to get the video from the edge-server which is closest to her
    - this means that the backend server must embed the CDN url instead of the database URL in the response page
- CDN's edge node will get the request for the CDN URL "cdn.akamai.com/a3b2fd5.mp4"
    - Edge node will check whether it has the file cached locally
        - if yes, then just serve the file
        - if not,
            - find the database URL
            - fetch the video from the database
            - cache it locally
            - serve it

Pankaj also wants to watch the video, and Pankaj is Upinta's neighbor

- Pankaj makes a request to the Facebook backend
- HTML file with CDN url
- Pankaj's browser tries to download from CDN edge server

- this time around, the edge server has the file cached!
    - because Upinta just watched it
- CDN server will just serve the file

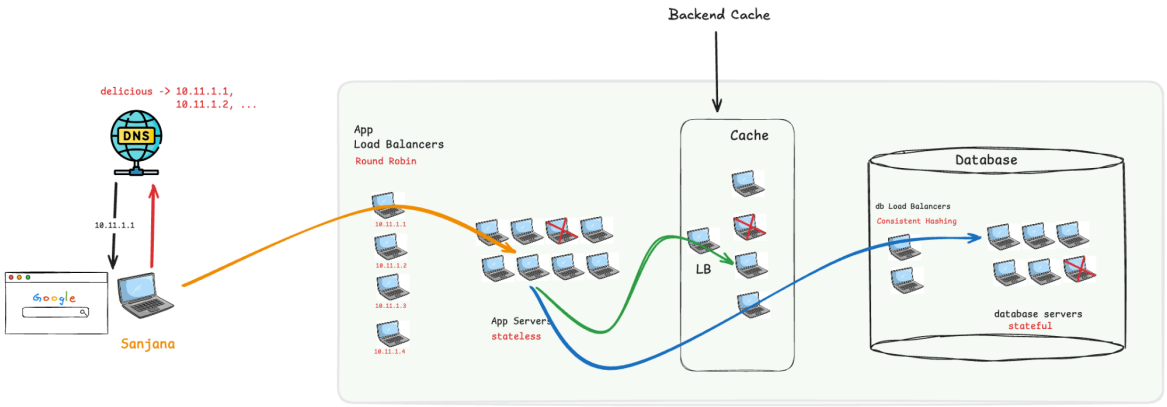
Q: Can the CDN be a bottleneck?

No! Because CDNs have millions of servers distributed all across the globe

(optional references)

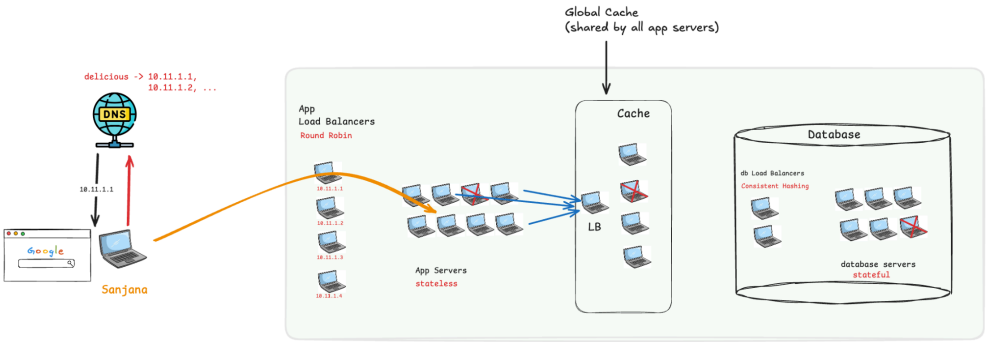
1. **Basics:**
    a. https://aws.amazon.com/what-is/cdn/
    b. https://www.akamai.com/glossary/what-is-a-cdn
    c. https://www.cloudflare.com/learning/cdn/what-is-a-cdn/
2. **AnyCast:** https://www.cloudflare.com/learning/cdn/glossary/anycast-network/
3. **Quick setup guide:** https://www.sumologic.com/blog/cdn-aws-cloudfront/
4. **CDN Cache Invalidation:**
    a. https://docs.aws.amazon.com/cloudfront/latest/APIReference/API_CacheBehavior.html
    b. https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Invalidation.html
5. **Some crazy CDN Stats (as of 2024)**
    a. Total CDN bandwidth (all CDNs combined) exceeds `10 Peta bits per second` (that's `10 million GBps`)
    b. Total CDN storage exceeds `1 Zetta bytes` (Giga < Tera < Peta < Exa < Zetta)
    c. Akamai alone handles `> 3 Exa bytes` per day!
    d. `70%` of all internet traffic is handled by CDNs

Backend Cache

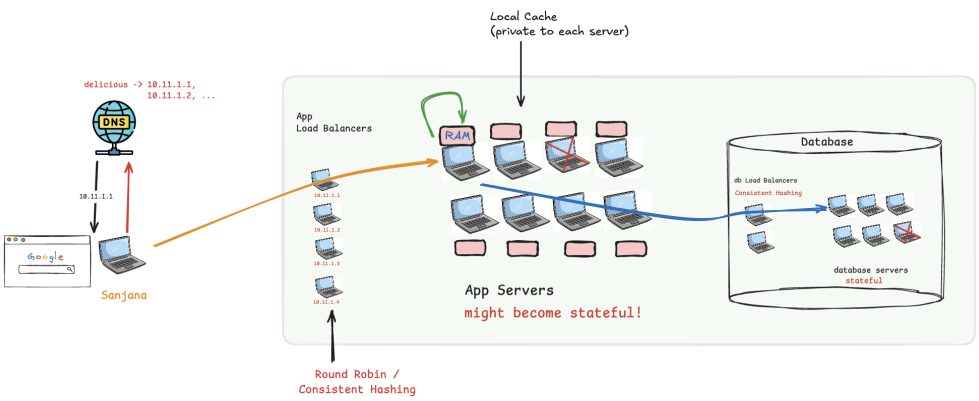


Local vs Global

- **Global:**
  - exists as a separate layer b/w the app server and the DB
  - all app servers can access this cache



- **Local:**
  - each app server can locally cache data inside its RAM/HDD
  - app servers are not allowed to access the cache of other app servers (the cache is **local & private** to each app server)
  - since the app servers are now storing data, it might make the app servers stateful (depending on how we use the cache)
    - which means that application LB might have to use consistent hashing



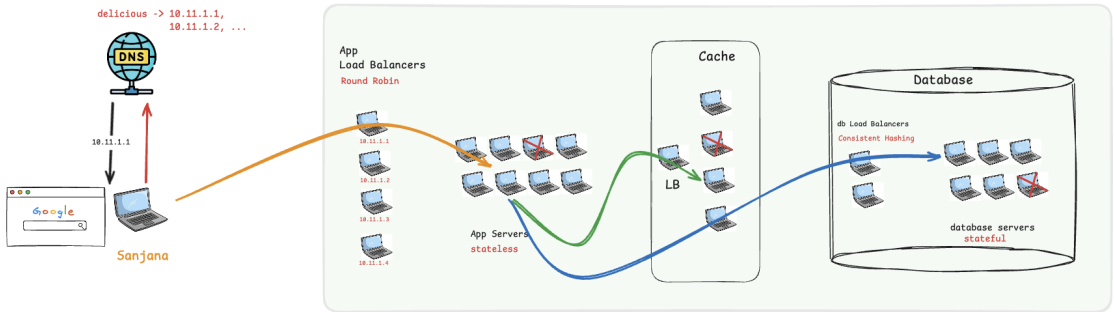
○

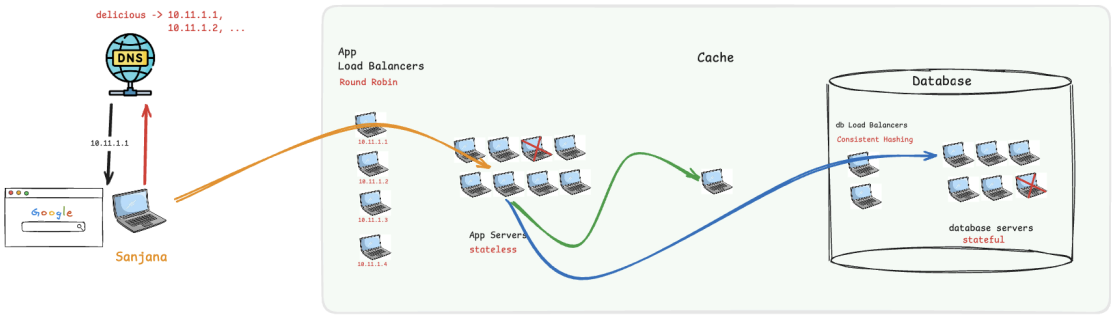
Single vs Distributed (only for Global caches)

Distributed cache: if a single cache server is not enough (we want to store more data in cache, or, we want to improve the read throughput), then we will use a distributed cache

Note: Single vs Distributed is only about global cache — because a local cache is inherently distributed (because each app server acts a cache, and there's multiple app servers)

Distributed Cache





Single Cache

Challenges with Caching

1. **Limited space:** caches are much smaller (compared to the DB)
    a. since it is small, it can get full
    b. if the cache is full, and you still want to insert a new entry in the cache - you must first **"evict"** something from the cache
    solution: **Cache Eviction**
2. **Stale data:** cache is not the source of truth - the database is the source of truth
    a. it is possible for the data to get updated in the database, but not in the cache
    b. then, the data inside the cache will be **"stale"** (old)
    c. you must detect when data is old, and you must **"invalidate"** the data (& remove it from the cache)
    solution: **Cache Invalidation**

Q: Should we do eviction, or should we do invalidation?

Both!
Both the cache invalidation algorithm and the cache eviction algorithm are always running together - each solves one problem.

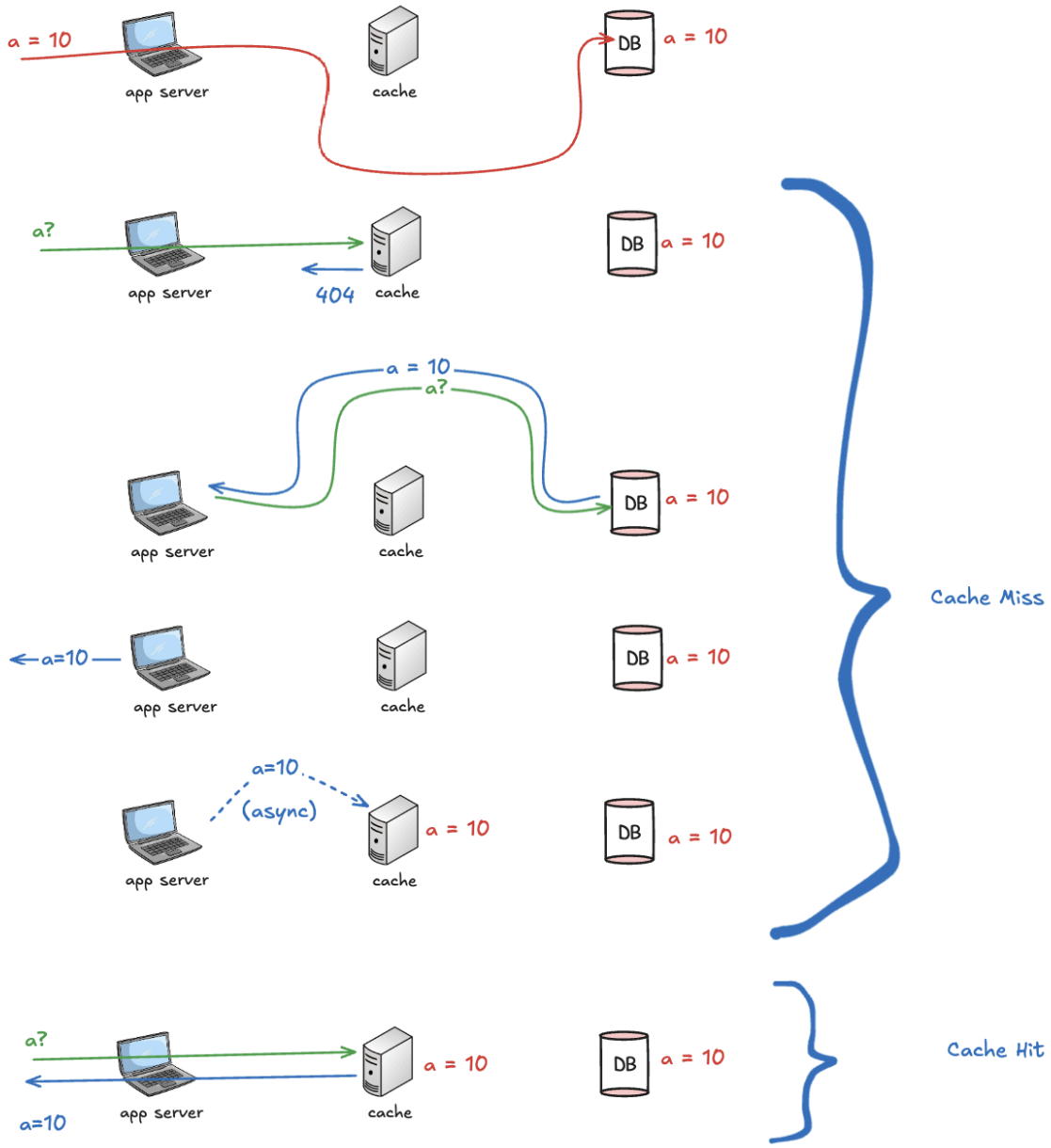
How cache reads/writes happen

typical read/write cycle

1. Write requests go directly to the database (typically)
    - not to the cache
2. Read request
    - first app servers checks if the value is in the cache
    - if present (Cache Hit)
        i. return the cached value
    - if not present (Cache Miss)
        i. app server fetches value from DB
        ii. returns to the user
        iii. (async) updates the value in the cache

Why doesn't the cache automatically fetch the data from the DB in case of a cache miss?

Separation of concerns. Business logic required to fetch data (auth / sorting / filtering / ...) resides in the app server - it is a bad idea to have the same business logic in multiple places.



Q: why can't we update the cache & the database together for each write?

because it's hard & slow — more on this later

Q: Can the cache itself fetch the data from the database?

Why does a cache miss have to happen in the above manner?
If the cache doesn't have some data, can't it itself fetch it from the database, store it, and then return it to the app server?

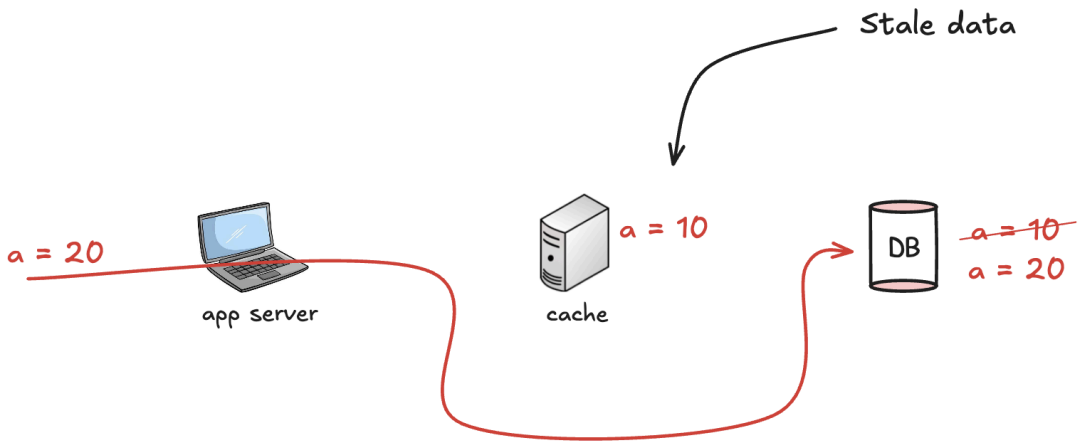
disclaimer: some people do it, but it is typically considered bad design (anti-pattern)

Reason: separation of concerns

1. business logic (what data to fetch, what tables to join, how to filter the data, how to validate it, how to sort it).. all of this lives inside the app server.
2. Cache server is supposed to be “dumb”
3. If we try to make the cache server intelligent, then our business logic will be split across two entities - app server & cache server
    - what if the two codebases are not in sync?
        - codebase logic can be stale in some place
    - cache server has to have a powerful CPU — to do all this slicing & dicing of data

Therefore, any fetching/updating responsibilities lies with the app server. Cache server just acts as dumb-storage.

Note: since any writes only go the DB, the data inside the cache can get stale



If a read now comes to the cache, the cache will tell you happily that the value is 10 — which is incorrect!
Note that because this will be a cache hit, the app-server will just return this incorrect value to the client — it won’t even check the database.

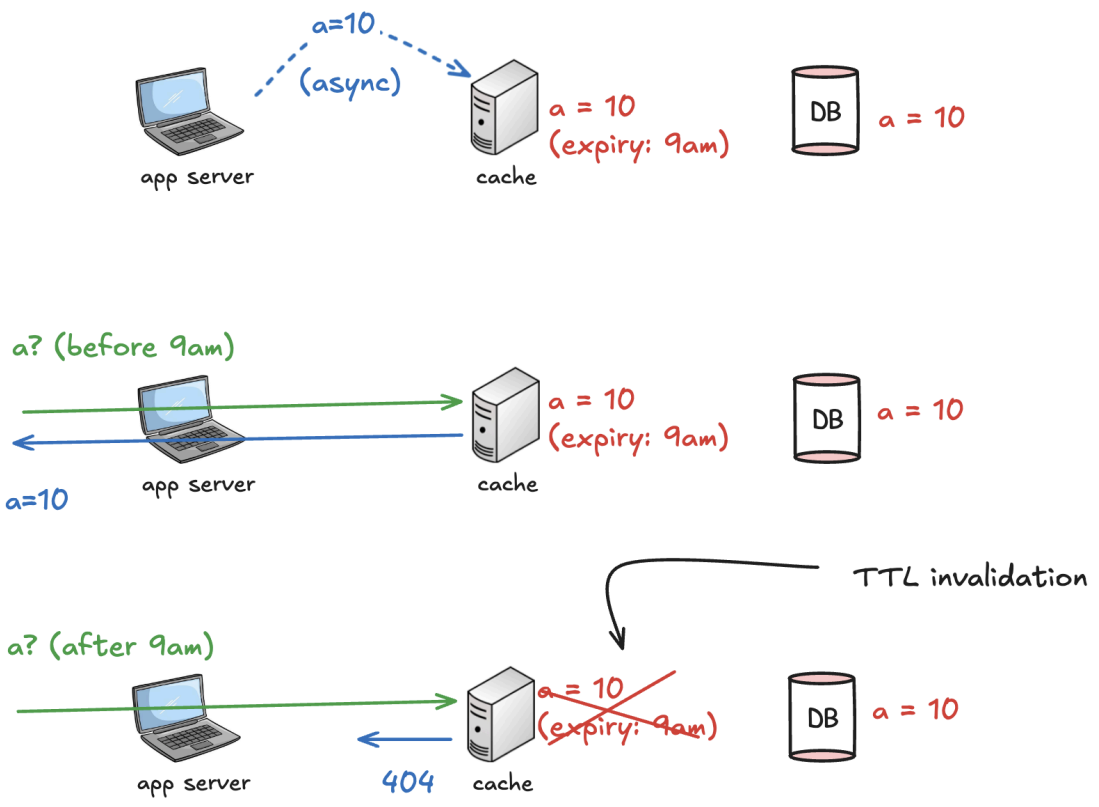
Cache Invalidation

Time to Live (TTL)

note: you might also see TTL be called a cache eviction strategy

Whenever you store data inside cache, apart from storing the value, you also store the expiry time.
Every cache entry has a fixed amount of time for which it is valid (TTL)
After the TTL expires, we assume that the value is stale, and we invalidate it.

Key	Value	Expiry
a	10	9 am
b	20	9.05 am



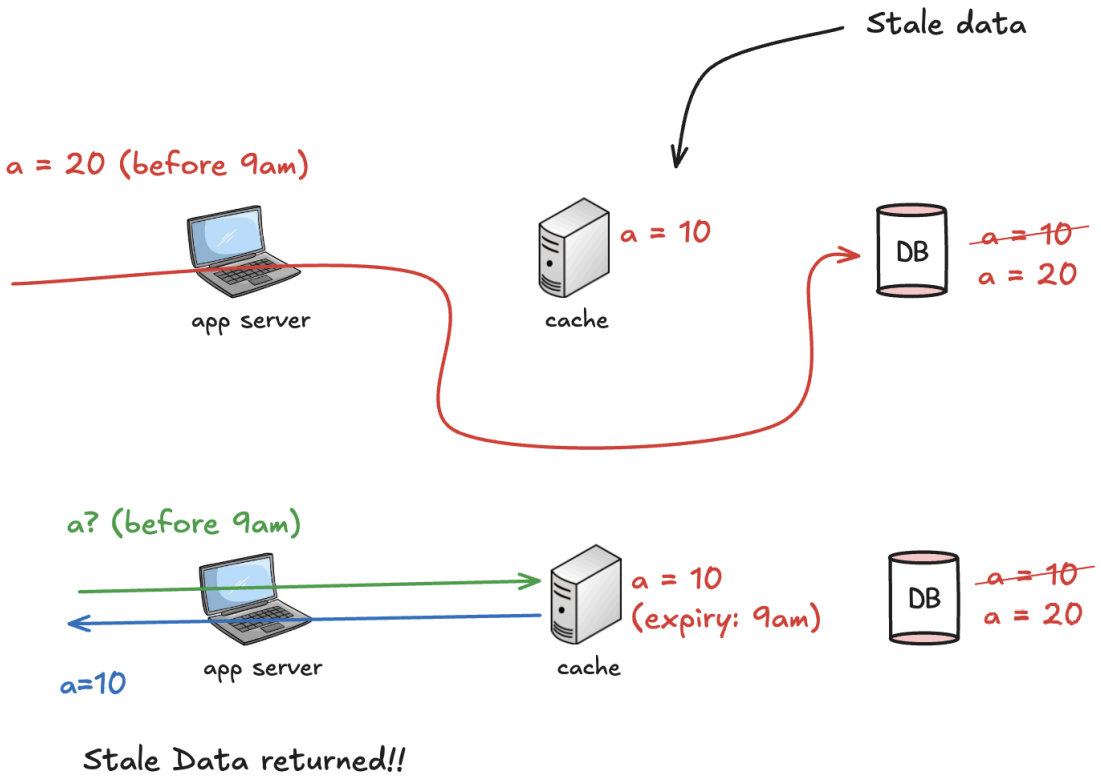
Q: What sort of consistency does TTL invalidation provide?

Q: Can the data served from the cache be stale?

What if we read stale data from the cache, before the value has a change to expire?

Yes, if an update happens in the database before the cache entry expires, then the subsequent reads (before expiry) will return the stale data from the cache.

So, TTL provides **"eventual consistency"**



Note: when the read request (after update) came, we first checked the cache (as we always do). Since we found the data in the cache, and the data had not expired (even though it was stale), we returned it to the user. We ended up serving stale data.

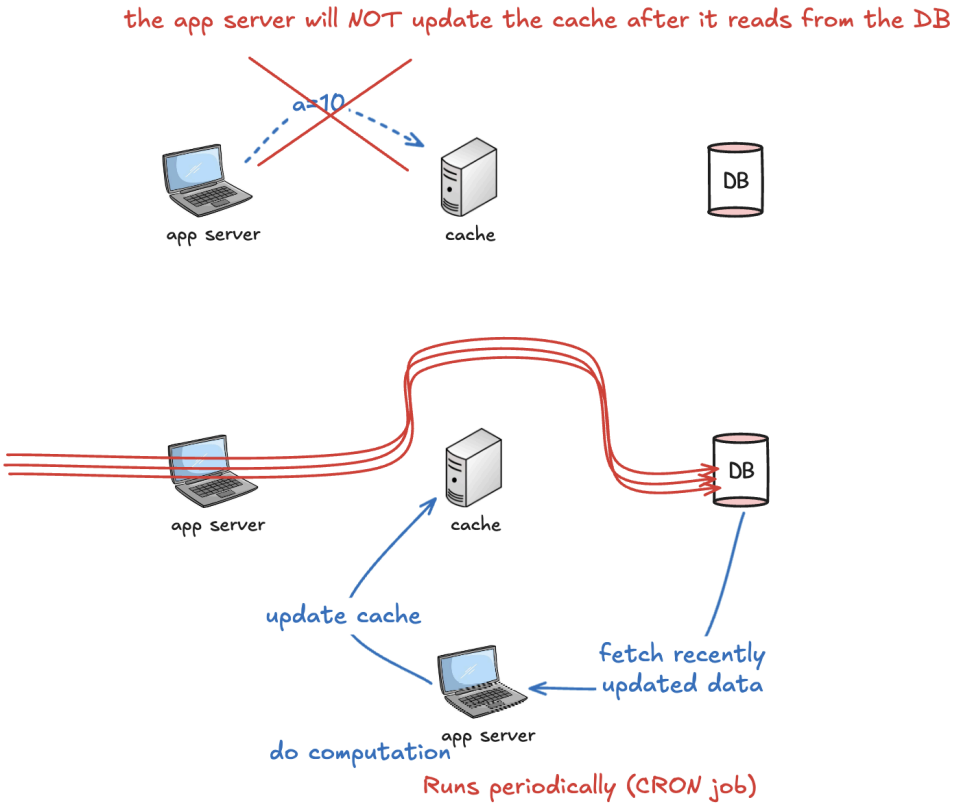
Write Around

very similar to TTL - as in it also provides "eventual consistency"
Key differences (compared to TTL)

1. **TTL:** works best when fetching an entry from the DB is easy, or requires no additional postprocessing/computation
2. **Write Around:** works best when fetching data from the DB is extremely expensive, or requires additional postprocessing/computation

In case of write around, there's a separate background process (running in a separate app server) that periodically fetches all recently updated data from DB, does the computation, and updates the cache in one go.

```
select * from ...
where updated_at > (now() - "1 hour")
```



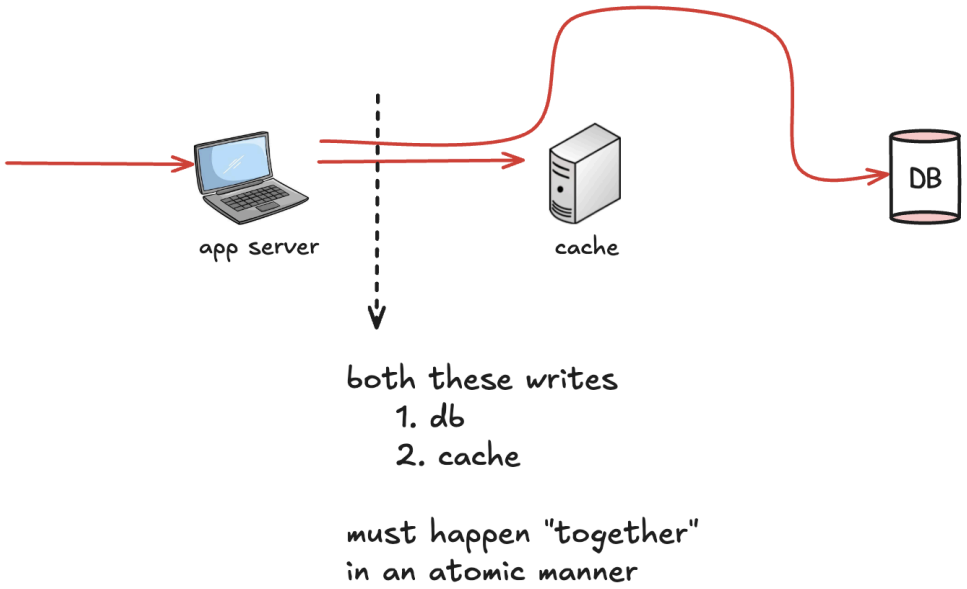
Q: Can Write Around serve stale data from the cache?

Yes. Until the background "cron job" runs again, any updates to the db means that the cache has stale data.

Write Through

In certain cases, we require **"immediate consistency" (aka strong consistency)** *(no stale reads, ever)*

The only way to achieve that is to ensure that whatever writes happen, happen at both the cache and the db together!



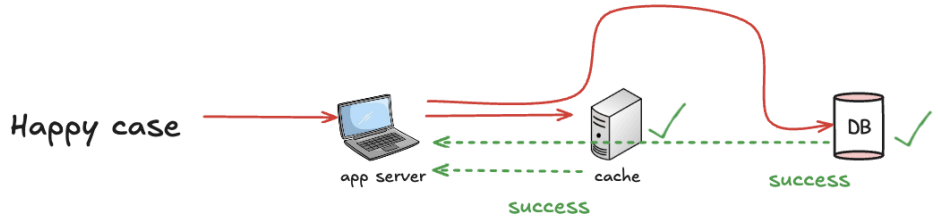
Write through cache is difficult - because ***ensuring atomicity across multiple servers is difficult***!

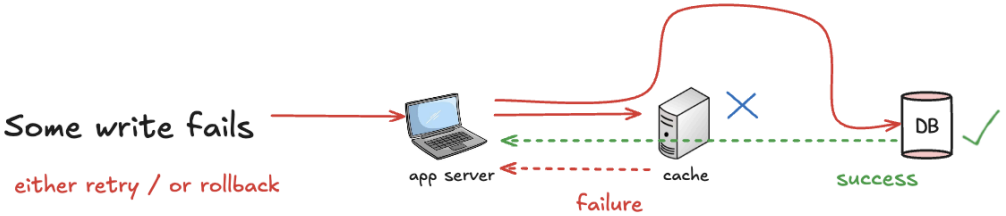
Single Server

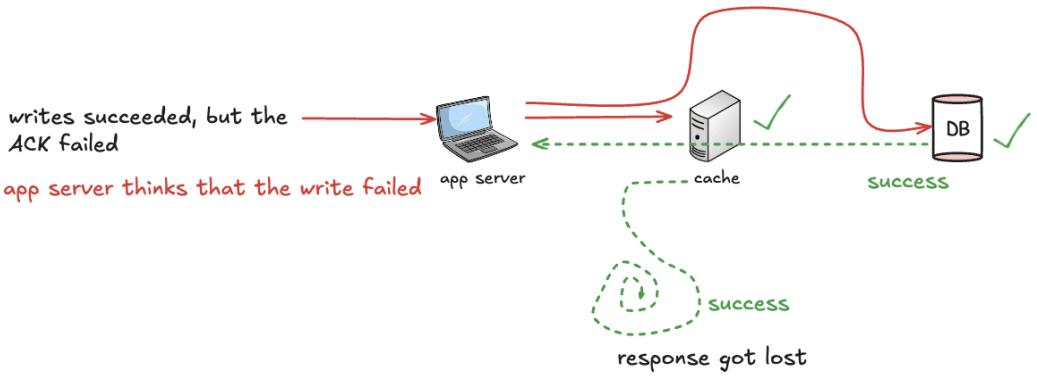
1. OS - thread synchronization (locks, semaphores, mutexes..)
2. Shared RAM
3. It is easy to ensure that two tasks happen atomically (either both, or neither)

Multiple Servers (cache & db)

1. No shared RAM
2. what if the write to DB succeeds but the write to the cache fails?
    a. either retry the write to cache (user will have to wait - high latency)
    b. or, rollback the DB write (transactions)
3. what if the write to DB succeeds, the write to cache succeeds, but the response from the cache fails?
    a. app server thinks that the write failed somewhere, so it will try to rollback/retry
4. what if the rollback fails
5. what if the ACK for the rollback fails?







2 Phase Commit - protocol used to perform atomic transactions in a distributed setting

- slow (high latency)
- hard to implement

Optional Readings

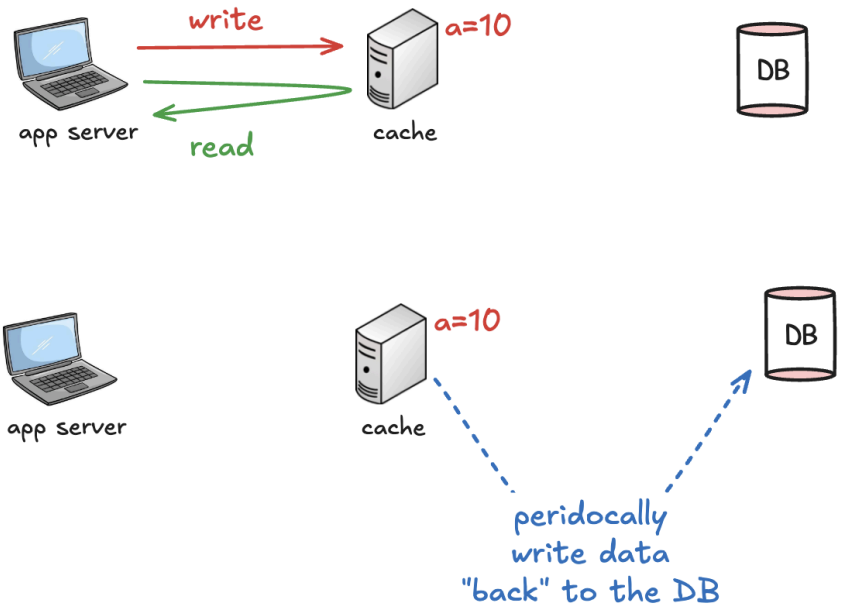
1. Theory: https://www.youtube.com/watch?v=-_rdWB9hN1c
2. Explanation 1: https://www.youtube.com/watch?v=eltn4x788UM
3. Explanation 2: https://www.youtube.com/watch?v=7FgU1D4EnpQ
4. Implementation: https://www.youtube.com/watch?v=oMhESvU87jM

Note: if the cache is local, you will ALWAYS use immediate consistency (write through). Because the cache is "local" (RAM/HDD) and it is guaranteed that writes always succeed.

Write Back

It provides no consistency guarantees.
In fact, write back cache **can lead to Data Loss!**

Writes do NOT go to the database - instead, writes go to the cache
Periodically, you write the data back from the cache to the database



Q: How can there be data loss?

If the cache server crashes with unsynced changes, then the data is permanently lost.

- cache is volatile, it doesn't store data on disk
- DB is persistent, but the cache server crashed with new data that had not been synced in the db yet

Q: Why would we ever use it?

Pros:

1. Any read/write is happening at cache
2. Any read/write will be insanely fast! (100,000+ reads/writes per second)

In scenarios where

1. the individual data points are not important
    a. when trends matter more than individual data points

b. analytics

2. extremely high write throughput is required

Famous Joke

There are only **3** difficult things in computer science

1. Naming variables
4. Thread synchronization
2. Cache Invalidation
3. Off by one errors

Cache Eviction

If you've just started the cache server, it is initially empty.

Do you want your cache server to be empty?
No! You want the cache to be full, because otherwise you're wasting much-needed cache. You want to cache as much data as possible!

As the operations continue, the cache server will get full — it will always remain full from this point onwards (unless it crashes & restarts)

But, you still want to cache more data into it (even though it is full) — this means, you first have to get rid of something that is already cached to make space for new data.

Cache Eviction algorithm decides which data to "kick-out" to make space for new data.

How do you decide which eviction policy to use?

1. just use LRU
    - no need to think at all — LRU works well for all usecases
    - it might not be the perfect choice, but it will work well enough
2. only if you've a lot of extra time, and you've already optimized everything else
    - you can implement and run all the various eviction strategies
    - measure which one is performing the best in practice
    - *(never guess)*

First in First Out (FIFO)

Last in First Out (LIFO)

Least Recently Used (LRU)

99% cases

H/W: read about

1. temporal locality of data
    - if some data has been accessed right now, then it is highly likely, that it will access again in the near future
2. spatial locality of data
    - if some data has been accessed right now, then it is highly likely, that “nearby” data will be accessed in the near future
    - [a, b, **c**, **d**, **f**, e, g, r, …]

Least Frequently Used (LFU)

Most Recently Used (MRU)

Complete opposite of LRU.

Bad for 99% cases!

H/W: find a valid use-case of the MRU eviction policy